TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ

BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐIỆN TOÁN ĐÁM MÂY

Đề tài 23

THIẾT KẾ VÀ KIỂM CHỨNG KẾ HOẠCH SAO LƯU PHỤC HỒI THẢM HỌA CHO ỨNG DỤNG CLOUD NATIVE

Mã lớp học phần: 012012303904

Nhóm: 23

Sinh viên thực hiện:

STT	Họ và tên	Mã số sinh viên
1	Hoàng Việt Quang	066206005044
2	Nguyễn Minh Thư	082306008557
3	Lê Thị Lan Hương	052306007313

Thành phố Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

MỞ ĐẦU 1
Lý do chọn đề tài 1
Câu hỏi nghiên cứu và mục tiêu 1
Giả thuyết 1
Phạm vi và cấu trúc báo cáo 2
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT 3
1.1 Đánh giá ảnh hưởng nghiệp vụ, RPO và RTO 3
1.1.1 Đánh giá ảnh hưởng nghiệp vụ 3
1.1.2 RPO và RTO 3
1.1.3 Thời hạn giữ bản sao lưu 3
1.1.4 Quy tắc 3-2-1 4
1.2 Các chiến lược dự phòng và chi phí sẵn sàng 4
1.3 Sao lưu nhất quán, bất biến và mã hóa 5
1.3.1 Sao lưu nhất quán ở mức ứng dụng 5
1.3.2 Bản sao nhất quán của cơ sở dữ liệu 5
1.3.3 Bản sao không thể sửa, xóa 5
1.3.4 Mã hóa bản sao lưu 6
1.4 Runbook, quyền kích hoạt, truyền thông và kiểm thử định kỳ 6
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ THÍ NGHIỆM 7
2.1 Ứng dụng được bảo vệ 7
2.2 Đánh giá ảnh hưởng và mục tiêu khôi phục 7
2.3 Chọn chiến lược dự phòng 8
2.4 Kiến trúc và hạ tầng 8
2.5 Sao lưu ba tầng và chính sách giữ 10
2.6 Mã hóa và quyền truy cập 11
2.7 Theo dõi sao lưu và diễn tập tự động 11
2.8 Kịch bản diễn tập 12
2.9 Phương pháp đo và kiểm tra dữ liệu 13
2.10 Runbook, quyền kích hoạt và truyền thông 14
CHƯƠNG 3: KẾT QUẢ QUAN SÁT 16
3.1 Chạy thử hạ tầng 16
3.2 Vòng thử 16
3.3 Vòng chính thức 17
3.4 Két chống xóa và khôi phục ngoài Azure 19
3.5 Bản thu nhỏ trên GitHub Actions 20
3.6 Chi phí thực tế 21

CHƯƠNG 4: DIỄN GIẢI VÀ GIỚI HẠN 22
4.1 Kiểm chứng giả thuyết 22
4.2 Sai lệch so với mục tiêu và điểm nghẽn 22
4.3 Runbook sau diễn tập 23
4.4 Kế hoạch kiểm thử lại 24
4.5 Chi phí sẵn sàng và lựa chọn chiến lược 25
4.6 Rủi ro bảo mật còn lại 25
4.7 Giới hạn của đồ án 26
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 27
5.1 Kết luận 27
5.2 Hướng phát triển 27
TÀI LIỆU THAM KHẢO 28
PHẦN BỔ SUNG SẢN PHẨM 31
PHỤ LỤC A: RUNBOOK SAU DIỄN TẬP 32
A.1 Khi nào kích hoạt 32
A.2 Chọn cách khôi phục 32
A.3 Các bước 32
A.4 Danh sách kiểm tra sau khôi phục 33
A.5 Lịch sử thay đổi 33
PHỤ LỤC B: BIÊN BẢN VÒNG THỬ 35

DANH MỤC CHỮ VIẾT TẮT

Chữ viết tắt	Ý nghĩa
API	Giao diện lập trình ứng dụng (application programming interface)
BIA	Đánh giá ảnh hưởng nghiệp vụ (business impact analysis)
CNPG	CloudNativePG, toán tử quản lý PostgreSQL trên Kubernetes
DNS	Hệ thống phân giải tên miền (domain name system)
DR	Phục hồi sau thảm họa (disaster recovery)
IaC	Hạ tầng mô tả bằng mã (infrastructure as code)
MTD	Thời gian ngừng tối đa chấp nhận được (maximum tolerable downtime)
p95	Phân vị thứ 95
PITR	Khôi phục về một thời điểm (point-in-time recovery)
RPO	Mục tiêu điểm khôi phục (recovery point objective)
RTO	Mục tiêu thời gian khôi phục (recovery time objective)
SAS	Mã truy cập có chữ ký của Azure Storage (shared access signature)
TT&E	Kiểm thử, huấn luyện và diễn tập (test, training and exercise)
TTL	Thời gian bản ghi DNS được ghi nhớ (time to live)
UTC	Giờ phối hợp quốc tế (coordinated universal time)
WAL	Nhật ký ghi trước của PostgreSQL (write-ahead log)
WORM	Ghi một lần, đọc nhiều lần (write once, read many)

DANH MỤC BẢNG

Bảng 1.1. So sánh các chiến lược dự phòng, tổng hợp từ [5] 5
Bảng 2.1. Đánh giá ảnh hưởng và mục tiêu RPO, RTO 7
Bảng 2.2. Dữ liệu được sao lưu, cách sao lưu và thời hạn giữ.....10
Bảng 2.3. Quyền tối thiểu của từng thành phần 11
Bảng 2.4. Bảy kịch bản diễn tập.....12
Bảng 3.1. Kết quả chạy thử hạ tầng trên Azure (24/10 đến 31/10/2026).....16
Bảng 3.2. Các lần không đạt ở vòng thử 17
Bảng 3.3. Kết quả vòng chính thức trên Azure 17
Bảng 3.4. Thời gian từng bước của lần có RTO trung vị (s).....19
Bảng 3.5. Kết quả bản thu nhỏ trên GitHub Actions 20
Bảng 4.1. Chi phí sẵn sàng mỗi tháng và RTO đo được 25
Bảng A.1. Chọn cách khôi phục theo tình huống.....32
Bảng A.2. Danh sách kiểm tra sau khôi phục 33
Bảng A.3. Lịch sử thay đổi runbook sau diễn tập 33
Bảng B.1. Kết quả vòng thử trên Azure 35

DANH MỤC HÌNH

Hình 2.1. Kiến trúc hệ thống: hai cụm ở Hàn Quốc, kho sao lưu và két ở Nhật, bản thứ ba ở máy chủ ngoài Azure 9
Hình 2.2. Trình tự chuyển sang dự phòng ấm và dự phòng nguội.....13
Hình 3.1. RTO và RPO của các lần đạt ở vòng chính thức trên Azure. Mỗi chấm là một lần, vạch đen là trung vị, trong ngoặc là số lần đạt trên số lần chạy; tải 1 bài/s, bộ mẫu 1.026 bài. Đơn vị: giây 18
Hình 3.2. Thời gian từng bước của lần đạt có RTO trung vị ở mỗi tình huống, vòng chính thức trên Azure, một lần mỗi tình huống. Đơn vị: giây 19
Hình 3.3. RTO của bản thu nhỏ trên GitHub Actions, máy 4 nhân, 30 lần mỗi tình huống (D2: 29 lần). Mỗi chấm là một lần, vạch đen là trung vị. Không dựng máy ảo, không đổi DNS nên không so trực tiếp với RTO trên Azure. Đơn vị: giây.....20

MỞ ĐẦU

Lý do chọn đề tài

Một ứng dụng cloud native [1] thường được dựng lại rất nhanh: mã nguồn nằm trong Git, hạ tầng mô tả bằng mã (IaC - infrastructure as code) bằng công cụ như Terraform [2], cấu hình triển khai là các tệp YAML. Điều đó dễ tạo cảm giác rằng sự cố nào cũng khôi phục được. Thực tế không như vậy. Dữ liệu người dùng, khóa bí mật, chứng chỉ và trình tự các bước khôi phục không tự nằm trong Git. Một bản sao lưu chưa từng được khôi phục thử chỉ là một giả định; Google tóm tắt điều này bằng nhận xét rằng không ai thật sự cần sao lưu, người ta cần khôi phục [3].

Kế hoạch phục hồi sau thảm họa (DR - disaster recovery) thường được viết ra với hai con số: lượng dữ liệu được phép mất (RPO) và thời gian được phép ngừng dịch vụ (RTO) [4]. Nếu kế hoạch chỉ nằm trên giấy, hai con số này chưa được kiểm chứng. Đề tài 23 đặt đúng câu hỏi đó, nên nhóm chọn đề tài để trả lời nó bằng số đo thật trên một hệ thống thật.

Câu hỏi nghiên cứu và mục tiêu

Câu hỏi nghiên cứu: kế hoạch phục hồi có thực sự đạt RPO và RTO hay không, nếu nó chỉ tồn tại trên tài liệu mà chưa thử khôi phục ứng dụng, cấu hình và dữ liệu?

Để trả lời, nhóm đặt ba mục tiêu theo đề:

- Phân tích tác động nghiệp vụ và đặt RPO, RTO cho từng thành phần của một web đăng bài kèm ảnh.
- Triển khai sao lưu tách miền lỗi cho dữ liệu và cấu hình triển khai: kho sao lưu ở vùng Azure khác, két chống xóa có mã hóa, và một bản ngoài Azure.
- Diễn tập mất namespace, mất cụm và mất cả nhóm tài nguyên chính, đo thời gian từng bước và kiểm tra dữ liệu sau khôi phục bằng công cụ tự động.

Giả thuyết

Năm giả thuyết dưới đây được đặt trước khi diễn tập. Kết quả bác bỏ cũng là kết quả hợp lệ.

- H1: dự phòng nguội (chỉ giữ bản sao lưu, sự cố xảy ra mới dựng hạ tầng) đưa web hoạt động trở lại trong vòng 60 phút.
- H2: dự phòng ấm (một cụm dự phòng chạy sẵn và liên tục nhận dữ liệu) đưa web hoạt động trở lại trong vòng 10 phút.
- H3: dữ liệu mất không vượt RPO của từng thành phần.
- H4: runbook chưa từng diễn tập sẽ có lỗi hoặc thiếu bước ở lần chạy đầu.
- H5: dữ liệu khôi phục được từ két chống xóa và từ bản ngoài Azure mà không cần kho sao lưu chính.

Phạm vi và cấu trúc báo cáo

Hệ thống chạy trên tài khoản Azure for Students và một bản thu nhỏ chạy trên GitHub Actions. Đồ án không làm quy trình quay về cụm cũ sau sự cố (failback), không chạy hai cụm cùng phục vụ (active-active) và không làm sẵn sàng cao bên trong một cụm.

Chương 1 trình bày cơ sở lý thuyết. Chương 2 mô tả thiết kế hệ thống và thiết kế thí nghiệm. Chương 3 báo cáo kết quả quan sát, không kèm diễn giải. Chương 4 diễn giải kết quả, phân tích điểm nghẽn, chi phí và giới hạn. Chương 5 kết luận. Phụ lục A là runbook sau diễn tập và danh sách kiểm tra; phụ lục B là biên bản vòng thử.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1 Đánh giá ảnh hưởng nghiệp vụ, RPO và RTO

1.1.1 Đánh giá ảnh hưởng nghiệp vụ

Đánh giá ảnh hưởng nghiệp vụ (BIA - business impact analysis) là bước đầu tiên của lập kế hoạch dự phòng trong NIST SP 800-34 [4]. Theo NIST, BIA gồm ba bước: xác định các quy trình nghiệp vụ mà hệ thống hỗ trợ và mức độ nghiêm trọng khi chúng bị gián đoạn, xác định tài nguyên cần để khôi phục, và xác định thứ tự ưu tiên khôi phục các tài nguyên đó [4]. Kết quả của BIA là các mục tiêu khôi phục cho từng thành phần, làm căn cứ chọn chiến lược và ước tính chi phí.

NIST dùng thêm khái niệm thời gian ngừng tối đa chấp nhận được (MTD - maximum tolerable downtime): khoảng thời gian dài nhất nghiệp vụ chịu được trước khi thiệt hại không thể chấp nhận. RTO phải nhỏ hơn MTD, vì sau khi hệ thống chạy lại vẫn còn việc kiểm tra và xử lý dữ liệu tồn đọng [4].

1.1.2 RPO và RTO

RPO (recovery point objective) là mốc thời gian mà dữ liệu phải khôi phục về được, tính ngược từ lúc sự cố. RPO bằng 5 phút nghĩa là chấp nhận mất tối đa 5 phút dữ liệu cuối [4]. Tần suất sao lưu quyết định RPO đạt được [5]: sao lưu mỗi ngày một lần thì RPO không thể nhỏ hơn một ngày.

RTO (recovery time objective) là thời gian tối đa từ lúc sự cố tới lúc dịch vụ phục vụ lại [4]. RTO của cả hệ thống không chỉ gồm thời gian chép dữ liệu về, mà còn gồm phát hiện sự cố, ra quyết định, dựng hạ tầng, khôi phục dữ liệu và chuyển lưu lượng. Đề tài đo riêng từng phần này để biết thời gian thật sự đi đâu.

1.1.3 Thời hạn giữ bản sao lưu

NIST yêu cầu chính sách sao lưu quy định tần suất, phạm vi, nơi lưu bản sao và chu kỳ luân chuyển phương tiện lưu trữ [4]. Chính sách giữ bản sao lưu (retention) là phần quy định mỗi loại bản sao được giữ bao lâu và bao nhiêu bản. Giữ quá ngắn thì khi phát hiện dữ liệu hỏng muộn, mọi bản sao đều đã chứa dữ liệu hỏng; giữ quá dài thì tốn chi phí lưu trữ. Với cơ sở dữ liệu có nhật ký giao dịch, retention còn quyết định khoảng thời gian có thể khôi phục về một thời điểm bất kỳ (PITR - point-in-time recovery) [6].

1.1.4 Quy tắc 3-2-1

Quy tắc 3-2-1 được Peter Krogh trình bày trong sách về quản lý ảnh số [7] và được US-CERT khuyến nghị [8]: giữ ba bản của dữ liệu quan trọng (một bản chính và hai bản sao lưu), trên hai loại phương tiện khác nhau, trong đó một bản ở nơi khác. Đồ án áp dụng quy tắc này trên đám mây bằng cách hiểu "loại phương tiện" là miền lỗi (failure domain) khác nhau: tài khoản lưu trữ khác, vùng khác, hoặc nhà cung cấp khác. Bản ở nơi khác phòng trường hợp mất cả một vùng. Để phòng mã độc tống tiền, CISA khuyến nghị giữ bản sao lưu ngoại tuyến, có mã hóa và kiểm tra định kỳ khả năng khôi phục [9].

1.2 Các chiến lược dự phòng và chi phí sẵn sàng

Các chiến lược DR khác nhau ở chỗ phần nào của hệ thống được dựng sẵn trước sự cố. Dựng sẵn càng nhiều thì RTO càng nhỏ và chi phí sẵn sàng (chi phí phải trả cả khi không có sự cố) càng lớn [5].

- Sao lưu và khôi phục (backup and restore), trong báo cáo gọi tắt là dự phòng nguội: chỉ có bản sao lưu. Khi sự cố xảy ra mới dựng hạ tầng và khôi phục. Chi phí thấp nhất, RTO dài nhất trong các chiến lược.
- Đèn mồi (pilot light): phần lõi như cơ sở dữ liệu bản sao chạy sẵn ở nơi dự phòng, phần còn lại tắt và chỉ bật khi cần.
- Dự phòng ấm (warm standby): một bản thu nhỏ của toàn hệ thống chạy sẵn và liên tục nhận dữ liệu, sự cố xảy ra thì nâng lên phục vụ. RTO tính bằng phút.
- Chủ động - bị động (active-passive): khái niệm chung cho ba cách trên, trong đó một nơi phục vụ người dùng, nơi kia chờ.
- Chủ động - chủ động (active-active): nhiều nơi cùng phục vụ, mất một nơi thì nơi còn lại gánh toàn bộ. RTO gần bằng 0 với phần lớn sự cố, nhưng là cách phức tạp và tốn kém nhất, và phải chọn cách ghi để tránh xung đột khi nhiều nơi cùng ghi [5].

Bảng 1.1. So sánh các chiến lược dự phòng, tổng hợp từ [5]

Chiến lược	Phần chạy sẵn	RTO tương đối	Chi phí sẵn sàng
Sao lưu và khôi phục	Chỉ bản sao lưu	Dài nhất	Thấp nhất, chủ yếu là tiền lưu trữ
Đèn mồi	Phần lõi, như cơ sở dữ liệu bản sao	Ngắn hơn	Thấp
Dự phòng ấm	Bản thu nhỏ của toàn hệ thống	Ngắn	Trung bình
Chủ động - chủ động	Toàn bộ hệ thống ở nhiều nơi	Gần 0	Cao nhất

1.3 Sao lưu nhất quán, bất biến và mã hóa

1.3.1 Sao lưu nhất quán ở mức ứng dụng

Một bản chụp đĩa (snapshot) lấy khi ứng dụng đang chạy chỉ có tính nhất quán như khi mất điện (crash-consistent): chỉ dữ liệu đã nằm trên đĩa tại lúc chụp được giữ lại, còn dữ liệu trong bộ nhớ và các thao tác ghi đang dở thì mất. Sao lưu nhất quán ở mức ứng dụng (application-consistent) ghi nhận cả nội dung bộ nhớ và các thao tác ghi đang chờ [10], thường bằng cách yêu cầu ứng dụng đẩy hết bộ đệm và tạm dừng ghi trước khi chụp. Velero hỗ trợ việc này bằng các lệnh (hook) chạy trong container của pod trước và sau khi sao lưu pod đó [11].

1.3.2 Bản sao nhất quán của cơ sở dữ liệu

Với PostgreSQL, cách được tài liệu chính thức khuyến nghị không phải là chụp đĩa mà là kết hợp một bản sao lưu đầy đủ (base backup) với việc lưu liên tục nhật ký ghi trước (WAL - write-ahead log) [6]. Khôi phục bằng cách nạp bản đầy đủ rồi phát lại WAL tới thời điểm mong muốn, nên có thể dừng ở đúng một giây trước sự cố. Bản xuất logic bằng `pg_dump` cũng nhất quán kể cả khi cơ sở dữ liệu đang được dùng, nhưng chỉ cho một mốc duy nhất, và tài liệu PostgreSQL lưu ý nó thường không phải cách sao lưu định kỳ chính cho hệ thống thật [12]. CloudNativePG (CNPG) tự động hóa cả hai việc này trên Kubernetes [13].

1.3.3 Bản sao không thể sửa, xóa

Bản sao bất biến (immutable backup) được lưu theo chế độ ghi một lần, đọc nhiều lần (WORM - write once, read many): trong thời hạn giữ, không ai sửa hay xóa được, kể cả quản trị viên. Azure Blob Storage hỗ trợ chính sách giữ theo thời gian; khi chính

sách đã khóa cứng (locked), không thể rút ngắn thời hạn [14]. Kể cả người có quyền quản trị tài khoản cũng không xóa được dữ liệu trong thời hạn giữ [14], nên bản sao vẫn còn khi mã độc tống tiền hoặc kẻ đánh cắp quyền quản trị cố xóa.

1.3.4 Mã hóa bản sao lưu

Bản sao lưu chứa toàn bộ dữ liệu nên cần được mã hóa [9]. Azure Storage tự mã hóa dữ liệu khi lưu bằng AES 256 bit, việc mã hóa và giải mã diễn ra trong suốt với người dùng [15]; vì vậy ai có quyền đọc tài khoản lưu trữ vẫn đọc được dữ liệu đã giải mã. Mã hóa phía người dùng trước khi tải lên, ví dụ bằng công cụ `age` [16], làm cho người giữ kho không đọc được bản sao nếu không có khóa riêng. Khi đó khóa giải mã trở thành thứ phải bảo vệ và phải còn dùng được khi mất cả hệ thống.

1.4 Runbook, quyền kích hoạt, truyền thông và kiểm thử định kỳ

Runbook là tài liệu ghi từng bước khôi phục. NIST khuyến nghị kế hoạch dùng danh sách kiểm tra và quy trình từng bước, để cả người không quen kế hoạch hay hệ thống vẫn thực hiện được [4]. Kế hoạch cũng phải nêu rõ ba nội dung [4]:

- Quyền kích hoạt: tiêu chí kích hoạt và người có thẩm quyền kích hoạt kế hoạch.
- Thông báo: báo cho ai, bằng cách nào, và làm gì khi không liên lạc được một người.
- Kiểm thử, huấn luyện và diễn tập (TT&E - test, training and exercise): kế hoạch phải được diễn tập định kỳ và sửa sau mỗi lần diễn tập, vì kiểm thử xác nhận khả năng khôi phục, huấn luyện giúp người thực hiện sẵn sàng, còn diễn tập lộ ra chỗ thiếu của kế hoạch [4], [17].

Bài giảng chương 4 của học phần [18] nêu hai nguyên tắc mà đồ án dùng làm căn cứ thiết kế: bản sao lưu chỉ có ý nghĩa khi khôi phục được đúng thời điểm cần, và sao lưu không thay thế nhân bản, nhân bản cũng không thay thế sao lưu. AWS cũng lưu ý nhân bản liên tục có thể không bảo vệ được dữ liệu trước lỗi làm hỏng dữ liệu hay hành vi xóa có chủ đích [5]. Vì vậy hệ thống của đồ án có cả hai: cụm bản sao cho dự phòng ấm và các bản sao lưu tách miền lỗi cho mọi tình huống.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ THÍ NGHIỆM

2.1 Ứng dụng được bảo vệ

Ứng dụng là một web đăng bài kèm ảnh. Người dùng xem danh sách, xem bài và tải ảnh; chỉ người có mã bí mật mới đăng được bài. Máy chủ API viết bằng Rust (Axum, sqlx), giao diện viết bằng Astro và Svelte, xuất thành trang tĩnh đặt trên GitHub Pages. Bài viết lưu trong PostgreSQL do CNPG quản lý; ảnh lưu trong Azure Blob Storage. Lúc đăng, API tính mã băm SHA-256 của ảnh và lưu vào bài viết, nhờ đó bước kiểm tra sau khôi phục so được từng ảnh với bản gốc.

Giao diện là tệp tĩnh nên vẫn tải được khi cụm chính mất. Vì vậy dấu hiệu khôi phục được đo ở API (`https://api.dr.qivarn.id.vn/healthz`), không đo ở trang giao diện.

2.2 Đánh giá ảnh hưởng và mục tiêu khôi phục

Bảng 2.1 là BIA rút gọn của web. Mục tiêu RPO, RTO được chốt trước vòng diễn tập thử và không đổi trong suốt đồ án.

Bảng 2.1. Đánh giá ảnh hưởng và mục tiêu RPO, RTO

Thành phần	Ảnh hưởng khi mất	RPO	RTO (nguội / ấm)	Cơ sở của RPO
Bài viết (PostgreSQL)	Không đọc, không đăng được bài; mất bài gần nhất	5 phút	15 phút / 2 phút	WAL gửi đi chậm nhất mỗi 60 s
Ảnh (Blob)	Bài hiện nhưng ảnh lỗi	5 phút	15 phút / 2 phút	Chép ảnh p95 129 s khi đo thử
Cấu hình Kubernetes, chứng chỉ	Không dựng được ứng dụng	1 giờ	15 phút / 0	Velero mỗi giờ, tệp gốc trong Git
Cả web, từ sự cố tới khi tên miền phục vụ lại	Người dùng không dùng được dịch vụ	5 phút	60 phút / 10 phút	Theo thành phần dữ liệu

RTO của từng thành phần chỉ tính thời gian khôi phục thành phần đó; các thành phần khôi phục song song. RTO của cả web cộng thêm phát hiện, quyết định, dựng hạ tầng và đổi DNS. Web là đồ án không có doanh thu, nên BIA dùng con số để chọn chiến lược theo chi phí (mục 4.5) thay vì ước tính thiệt hại thật.

2.3 Chọn chiến lược dự phòng

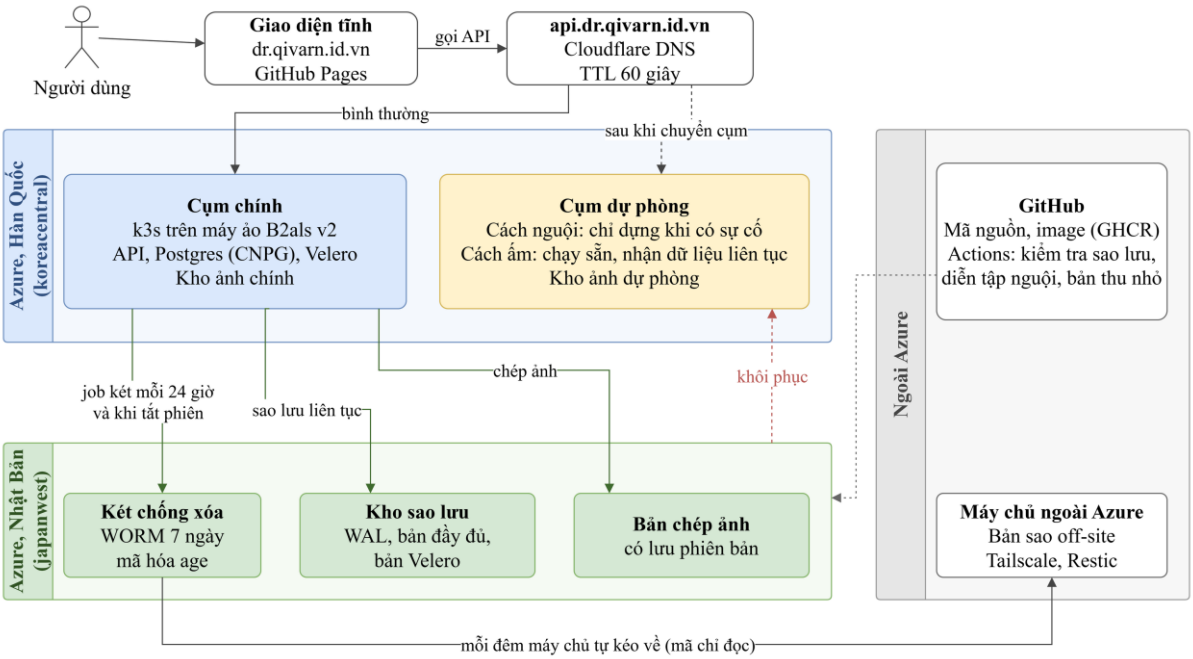
Đồ án so sánh hai chiến lược ở hai đầu của đánh đổi chi phí và thời gian: dự phòng nguội (sao lưu và khôi phục) và dự phòng ấm. Đèn mồi nằm giữa hai chiến lược này nên không làm riêng. Active-active bị loại vì cần hai cụm cùng ghi vào một cơ sở dữ liệu, ngoài khả năng của hạn mức 4 nhân CPU, và vì RTO gần 0 không trả lời được câu hỏi kế hoạch có đạt mục tiêu hay không.

Với dự phòng ấm, cụm dự phòng chạy sẵn một cụm Postgres bản sao (replica cluster) của CNPG [19], liên tục đọc WAL từ kho ở Nhật; kho ảnh dự phòng nhận ảnh qua tính năng chép đối tượng (object replication) của Azure [20]. Với dự phòng nguội, cụm dự phòng chưa tồn tại; khi có sự cố mới dựng bằng Terraform [2] rồi khôi phục từ kho ở Nhật.

2.4 Kiến trúc và hạ tầng

Hệ thống dùng ba nhóm tài nguyên Azure (Hình 2.1):

- `rg-dr-primary` ở `koreacentral`: máy ảo chạy cụm chính và tài khoản lưu trữ ảnh chính.
- `rg-dr-standby` ở `koreacentral`: máy ảo chạy cụm dự phòng và tài khoản lưu trữ ảnh dự phòng.
- `rg-dr-backup` ở `japanwest`: bốn tài khoản lưu trữ cho trạng thái Terraform, kho sao lưu (WAL, bản đầy đủ, Velero), bản chép ảnh, và két chống xóa.



Hình 2.1. Kiến trúc hệ thống: hai cụm ở Hàn Quốc, kho sao lưu và két ở Nhật, bản thứ ba ở máy chủ ngoài Azure

Các lựa chọn chính và lý do:

- k3s [21] trên máy ảo thay vì AKS. Tài khoản sinh viên chỉ có 4 nhân CPU, còn nhóm máy hệ thống (system node pool) của AKS cần máy ít nhất 4 nhân, ít nhất 2 máy và không hỗ trợ dòng B [22]. Mỗi cụm là một máy B2als v2 (2 nhân, 4 GB RAM) [23] cài k3s bằng cloud-init, hai cụm dùng đúng 4 nhân.
- Cả hai cụm ở `koreacentral` vì tài khoản chỉ cho tạo máy ảo ở vùng này; kho lưu trữ được phép đặt ở Nhật nên bản sao lưu nằm thật ở vùng khác. Tình huống mất vùng vì vậy được giả lập bằng cách xóa cả nhóm `rg-dr-primary`.
- DNS ở Cloudflare thay vì Azure DNS. Bản ghi A của `api.dr.qivarn.id.vn` có thời gian ghi nhớ (TTL) 60 s; script chuyển cụm gọi API của Cloudflare để đổi địa chỉ. Cách này không tốn tiền và bớt một chỗ phụ thuộc Azure.
- Không dùng MinIO như đề gợi ý vì kho mã nguồn mở của MinIO ghi rõ không còn được bảo trì và đã chuyển sang chỉ đọc từ ngày 25/04/2026 [24]; Azure Blob có sẵn khóa chống xóa [14].
- Hạ tầng viết bằng Terraform, phần mềm trong cụm cài bằng Helm [25], image của ứng dụng để trên GitHub Container Registry, tức là nằm ngoài Azure.

- Chứng chỉ HTTPS của Let's Encrypt chỉ xin một lần, sau đó được Velero sao lưu và khôi phục sang cụm mới, vì Let's Encrypt chỉ cấp tối đa 5 chứng chỉ cho cùng một tập tên miền trong 7 ngày [26].

Để tiết kiệm, cụm chỉ tồn tại trong phiên làm việc hoặc diễn tập. Script tắt phiên lấy một bản đầy đủ, một bản Velero, chạy job két rồi xóa máy ảo; chỉ các tài khoản lưu trữ được giữ lại. Mỗi lần bật phiên vì vậy cũng là một lần khôi phục thật từ kho ở Nhật.

2.5 Sao lưu ba tầng và chính sách giữ

Bảng 2.2 liệt kê dữ liệu được sao lưu, cách sao lưu và thời hạn giữ. Ba tầng bản sao thỏa quy tắc 3-2-1: bản đang chạy ở Hàn Quốc, bản ở hai tài khoản riêng tại Nhật (kho sao lưu và két), và bản ở máy chủ ngoài Azure.

Bảng 2.2. Dữ liệu được sao lưu, cách sao lưu và thời hạn giữ

Dữ liệu	Cách sao lưu	Thời hạn giữ
Bài viết	CNPG và Barman Cloud Plugin [27] gửi WAL sang kho ở Nhật với `archive_timeout` [6] 60 s; bản đầy đủ mỗi 6 giờ, trước mỗi lần diễn tập và lúc tắt phiên	7 ngày, khôi phục được về thời điểm bất kỳ trong khoảng đó
Ảnh	Object replication từ kho ảnh chính sang bản chép ở Nhật, có lưu phiên bản theo yêu cầu của tính năng này [20]	Phiên bản cũ xóa sau 7 ngày
Cấu hình Kubernetes, chứng chỉ	Velero mỗi giờ, lưu vào kho ở Nhật	3 ngày; bản trước diễn tập và lúc tắt phiên giữ 7 ngày
Gói két: `pg_dump`, ảnh mới, bản Velero	Job két lúc tắt phiên và mỗi 24 giờ, mã hóa bằng `age` rồi tải lên két WORM	7 ngày, chính sách khóa cứng trước vòng chính thức
Bản ngoài Azure	Máy chủ ngoài Azure mỗi đêm kéo gói mới từ két qua Tailscale [28], lưu bằng restic [29]	7 bản theo ngày, tối đa 3 GB
Hạ tầng, tệp cấu hình, image	Git và GitHub Container Registry	Lâu dài

Tầng hai và tầng ba lưu bản xuất `pg_dump` thay vì WAL vì bản xuất chỉ vài MB, khôi phục được vào bất kỳ Postgres nào mà không cần CNPG. Khi khôi phục bằng Velero, đối tượng `Cluster` của CNPG và ổ đĩa của Postgres được loại trừ: nếu để Velero dựng lại, CNPG sẽ tạo một cơ sở dữ liệu rỗng. Cơ sở dữ liệu luôn được khôi phục bằng CNPG từ kho ở Nhật.

2.6 Mã hóa và quyền truy cập

Dữ liệu được mã hóa khi truyền bằng HTTPS (Azure, người dùng) và WireGuard, lớp nền của Tailscale [28] (máy chủ ngoài Azure). Khi lưu, Azure tự mã hóa AES 256 bit [15]; gói trong két được mã hóa thêm bằng `age` [16] ngay trong cụm, trước khi rời khỏi cụm. Gói được mã hóa cho hai khóa công khai: khóa khôi phục của người phụ trách hạ tầng và khóa riêng cho bài diễn tập tự động. Khóa bí mật không nằm trên Azure, không nằm trong Git và không nằm trên máy chủ ngoài Azure.

Mỗi thành phần dùng một mã truy cập có chữ ký (SAS - shared access signature) riêng. SAS cho phép giới hạn chi tiết quyền truy cập vào tài khoản lưu trữ [30], nên mỗi mã chỉ có đúng quyền cần (Bảng 2.3). Nhờ đó, lộ mã của cụm cũng không xóa được két.

Bảng 2.3. Quyền tối thiểu của từng thành phần

Thành phần	Quyền trên kho lưu trữ
API	Đọc và ghi kho ảnh
CNPG và Velero	Ghi kho sao lưu
Job két	Chỉ tạo mới trong két, không ghi đè, không xóa
Máy chủ ngoài Azure	Chỉ đọc két
Kiểm tra sao lưu mỗi giờ trên GitHub	Chỉ liệt kê và đọc kho sao lưu
Diễn tập tự động mỗi ngày trên GitHub	Chỉ đọc kho sao lưu, bản chép ảnh và két

2.7 Theo dõi sao lưu và diễn tập tự động

Sao lưu hỏng mà không ai biết là rủi ro lớn nhất của một kế hoạch DR. Một workflow GitHub Actions chạy mỗi giờ, ngoài Azure, đọc kho ở Nhật bằng mã chỉ đọc và báo lỗi qua email khi WAL mới nhất cũ hơn 5 phút, bản Velero mới nhất cũ hơn 2 giờ, job két gần nhất thất bại, hoặc mã SAS còn dưới 7 ngày là hết hạn.

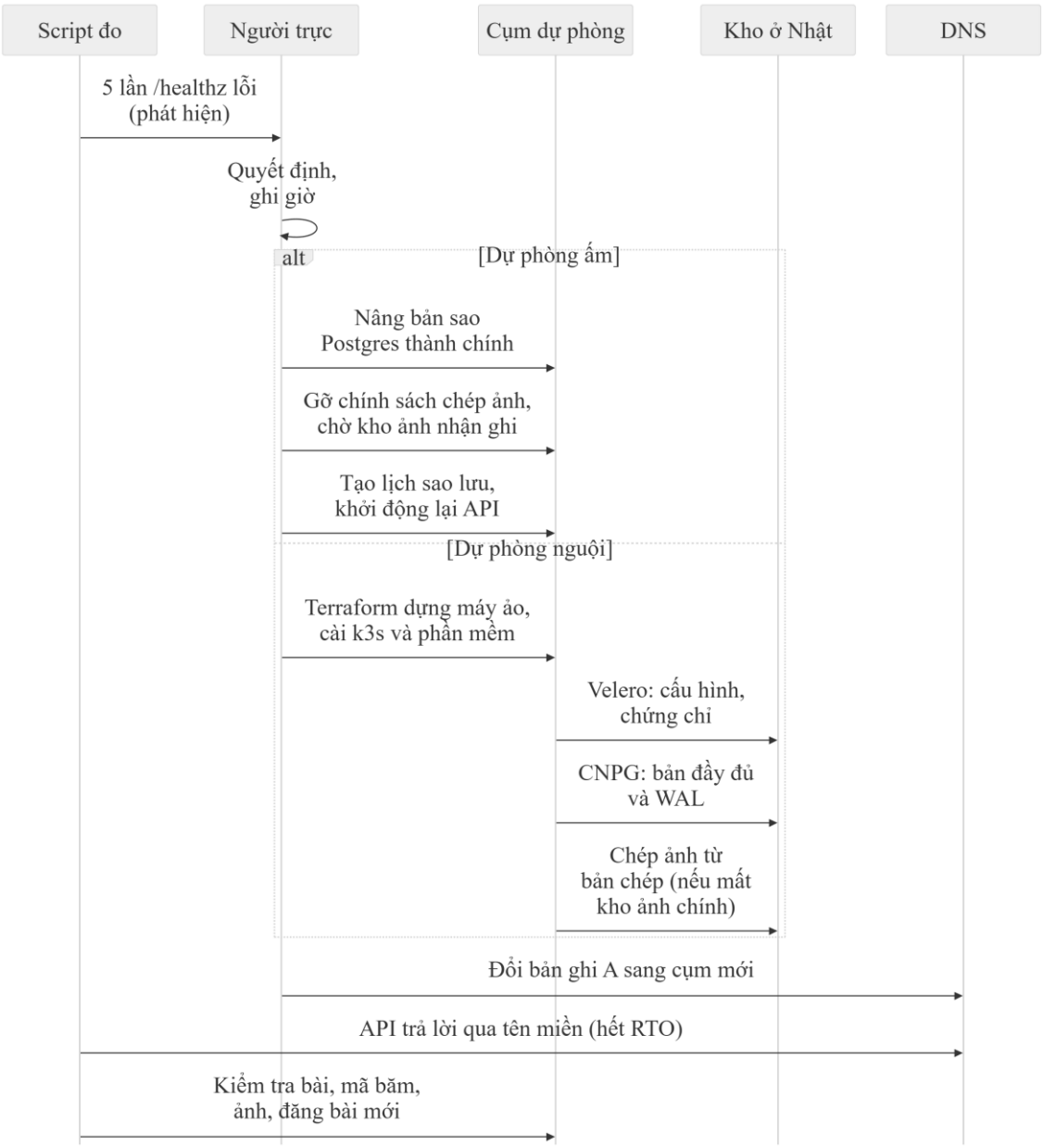
Một workflow khác chạy mỗi ngày, khôi phục thử bản sao lưu thật vào một cụm kind [31] trong máy ảo của GitHub: CNPG dựng cơ sở dữ liệu từ kho ở Nhật, Velero khôi phục cấu hình, đồng thời giải mã gói mới nhất trong két và khôi phục vào một Postgres riêng. Như vậy tầng một và tầng hai được khôi phục thử mỗi ngày, kể cả những ngày cụm trên Azure đã tắt.

2.8 Kịch bản diễn tập

Bảng 2.4. Bảy kịch bản diễn tập

Mã	Sự cố	Cách khôi phục	Cách gây sự cố
D1	Xóa nhầm namespace [32] của web	Khôi phục tại chỗ, đưa cơ sở dữ liệu về 1 s trước sự cố	`kubectl delete namespace dr-api`
D2	Mất cụm chính	Dự phòng nguội; kho ảnh chính còn nên dùng lại	Xóa máy ảo cụm chính
D3	Mất cụm chính	Dự phòng ấm	Xóa máy ảo cụm chính
D4	Mất cả nhóm `rg-dr-primary` (giả lập mất vùng)	Dự phòng nguội; chép ảnh từ bản chép ở Nhật	Xóa cả nhóm tài nguyên
D5	Mất cả nhóm `rg-dr-primary`	Dự phòng ấm	Xóa cả nhóm tài nguyên
D6	Kẻ có quyền quản trị muốn xóa bản sao lưu	Không cần khôi phục; két phải từ chối	Xóa, ghi đè gói trong két bằng khóa tài khoản và SAS
D7	Mất kho ở Nhật hoặc mất cả Azure	Khôi phục từ két và từ máy chủ ngoài Azure lên laptop	Khôi phục vào Postgres chạy bằng Docker trên laptop

Hình 2.2 là trình tự của hai cách chuyển dự phòng. Mỗi bước trong runbook là một lệnh tự ghi giờ bắt đầu, giờ kết thúc và kết quả vào tệp CSV.



Hình 2.2. Trình tự chuyển sang dự phòng ấm và dự phòng nguội

Diễn tập gồm hai vòng. Vòng thử chạy mỗi tình huống một lần để tìm lỗi của runbook, sửa sau mỗi lần. Vòng chính thức chạy D1 tới D5 mỗi tình huống ba lần, D6 và D7 mỗi tình huống một lần, trên cùng một phiên bản mã.

2.9 Phương pháp đo và kiểm tra dữ liệu

Công cụ `azure_drill` (Python) chạy trên laptop suốt lần diễn tập và làm bốn việc:

- Mỗi giây gọi `/healthz` qua tên miền và gọi thẳng vào từng cụm, nhờ đó tách được lúc API sống lại khỏi lúc DNS đổi xong. Đồng thời gọi một địa chỉ ổn định bên ngoài; giây nào mạng ở laptop mất thì bị loại.

- Mỗi giây đăng một bài có số thứ tự kèm một ảnh ngẫu nhiên. Bài nào API xác nhận đã nhận thì ghi số thứ tự, giờ và mã băm vào sổ.
- Coi 5 lần gọi lỗi liên tiếp là phát hiện sự cố, rồi chạy runbook và ghi thời gian sáu bước: phát hiện, quyết định, dựng hạ tầng, khôi phục dữ liệu, chuyển lưu lượng, kiểm tra. RTO tính từ lúc gây sự cố tới lúc API trả lời thành công qua tên miền.
- Sau khôi phục, kiểm tra theo danh sách ở phụ lục A: so số bài với sổ, so mã băm nội dung của toàn bộ bài viết, tải lại từng ảnh và so mã băm, thử đăng một bài mới.

RPO thực tế là khoảng từ bài cuối cùng còn lại tới lúc sự cố. Bài thiếu nằm ngoài RPO và ảnh hỏng được liệt kê riêng. Ảnh không tải được của bài đăng trong 300 s trước sự cố được tính là mất trong RPO ảnh, vì chép ảnh sang Nhật là bất đồng bộ; ảnh của bài cũ hơn hoặc ảnh sai mã băm là ảnh hỏng.

Để các lần đo so sánh được với nhau, trước mỗi lần diễn tập cơ sở dữ liệu được đưa về cùng một bộ mẫu 1.026 bài, và tải luôn là 1 bài mỗi giây. Trên Azure mỗi tình huống chỉ có ba lần nên báo cáo nêu giá trị nhỏ nhất, trung vị và lớn nhất, không suy luận thống kê. Bản thu nhỏ lặp 30 lần nên báo cáo nêu trung vị và khoảng tứ phân vị.

Bản thu nhỏ dựng hai cụm kind trong cùng một máy ảo GitHub Actions (4 nhân, khoảng 16 GB RAM), dùng SeaweedFS thay Azure Blob, cùng ứng dụng, cùng CNPG, Velero và cùng công cụ kiểm tra. Nó không có mất vùng, không có đổi DNS và không có object replication, nên chỉ chạy D1, D2, D3, D6. Số đo của bản thu nhỏ được báo cáo riêng, không trộn với Azure.

2.10 Runbook, quyền kích hoạt và truyền thông

Runbook (`docs/runbook.md`, bản sau diễn tập ở phụ lục A) chọn cách khôi phục theo tình huống: mất namespace thì khôi phục tại chỗ, mất máy chính thì chuyển sang dự phòng nguội hoặc dự phòng ấm tùy cụm dự phòng có đang chạy hay không, mất cả nhóm tài nguyên thì chuyển sang dự phòng nguội có chép ảnh từ Nhật hoặc sang dự phòng ấm.

Người phụ trách hạ tầng có quyền kích hoạt khôi phục; nếu không liên lạc được trong 15 phút thì thành viên được chỉ định thay quyết định. Người quyết định ghi giờ

quyết định. Truyền thông dùng nhóm Zalo của đồ án với ba mẫu tin có ghi giờ: bắt đầu sự cố, đã chuyển sang dự phòng, đã khôi phục xong. Khóa và mã bí mật nằm trong thư mục riêng trên máy người trực và trong trình quản lý mật khẩu; runbook chỉ ghi khóa nằm ở đâu, không ghi bản thân khóa.

CHƯƠNG 3: KẾT QUẢ QUAN SÁT

3.1 Chạy thử hạ tầng

Trước khi diễn tập, từng thành phần được chạy thử để chọn cấu hình (Bảng 3.1).

Bảng 3.1. Kết quả chạy thử hạ tầng trên Azure (24/10 đến 31/10/2026)

Hạng mục	Số đo	Quyết định
RAM của cụm k3s (B2als v2, 3.906 MB)	Rảnh 1.772 MB; tải 1 bài/s trong 1 giờ, đỉnh 1.797 MB (46%)	Giữ B2als v2
WAL sinh ra	Khoảng 111 MB mỗi ngày với `archive_timeout` 60 s	Giữ cấu hình
Chép 200 ảnh Hàn Quốc sang Nhật	Trung vị 52,7 s, p95 128,7 s, lớn nhất 139,2 s	RPO ảnh 5 phút
Dựng cụm mới	Cụm chính 2 phút 15 s, cụm dự phòng 1 phút 57 s	Dựng hạ tầng dưới 6 phút
Cụm Postgres bản sao	Sẵn sàng sau 72 s, theo kịp cụm chính sau 61 s, đổi vai thành chính 7 s	Dùng cho dự phòng ấm
Chứng chỉ Let's Encrypt	Velero khôi phục đúng chứng chỉ cũ sang cụm mới	Không xin chứng chỉ mới
Ảnh nhỏ AVIF	p95 1.340 ms, vượt ngưỡng 500 ms	Đổi sang WebP
Máy GitHub Actions cho bản thu nhỏ	Hai cụm kind dùng 3,45 GB, còn trống 4,48 GB	Đủ cho diễn tập tự động

3.2 Vòng thử

Vòng thử chạy từ 31/10 đến 04/11/2026, gồm 12 lần cho D1 đến D5. Bảy lần không đạt; mỗi lần lộ một lỗi khác nhau của runbook hoặc script (Bảng 3.2). Chi tiết từng lần ở phụ lục B.

Bảng 3.2. Các lần không đạt ở vòng thử

Lần	Bước hỏng	Lỗi lộ ra
D1-1	Khôi phục (910 s)	Cụm mới chưa có bản đầy đủ nào trong kho; Velero lấy nhầm bản của phiên trước nên API chạy image cũ
D1-2	Khôi phục (8 s)	Câu lệnh jsonpath chọn bản Velero sai cú pháp
D2-1	Dựng hạ tầng (2.490 s)	Máy ảo mới kẹt ở trạng thái `starting`, Azure trả `VMStartTimedOut` sau 41 phút
D2-2	Kiểm tra	Dữ liệu đủ nhưng không đăng được bài: script gỡ chính sách chép ảnh so sai ID nên kho ảnh dự phòng vẫn chỉ đọc
D2-3	Kiểm tra	50 ảnh thiếu ở kho dự phòng vì lần sửa trước gỡ chính sách mà không dựng lại
D4-1	Khôi phục (1.616 s)	Bước chép ảnh chờ mọi ảnh ở kho đích, kể cả ảnh không bao giờ tới
D4-2	Khôi phục (301 s)	Azure từ chối ghi ảnh vì chính sách chép ảnh cũ còn ở kho dự phòng

Năm lần đạt: D1-3 (RTO 74,0 s), D2-4 (451,5 s), D3-1 (247,2 s), D4-3 (588,4 s) và D5-1 (262,3 s). Lần D4-3 lúc chạy bị chấm không đạt vì 16 ảnh không tải được; cả 16 ảnh đều của bài đăng 17 s đến 32 s trước sự cố, chưa kịp chép sang Nhật. Luật chấm được sửa thành luật RPO ảnh ở mục 2.9, và D4-3 được chấm lại là đạt trên đúng dữ liệu đã khôi phục.

3.3 Vòng chính thức

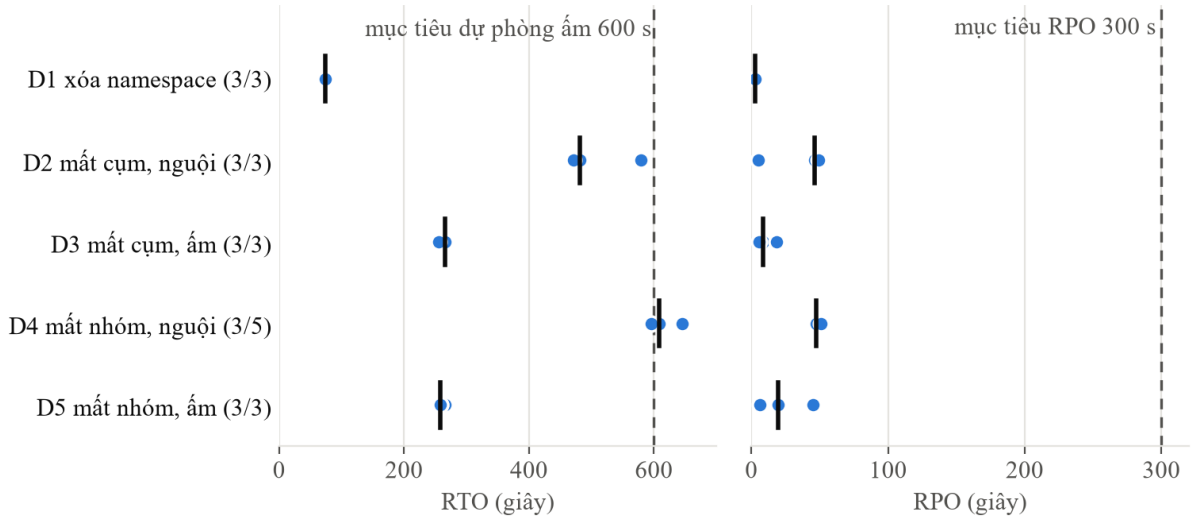
Vòng chính thức chạy ngày 07 và 08/11/2026 trên commit `850ac35`. Mọi lần đều được báo trước, chạy bằng `azure_drill` với tùy chọn tự quyết định nên bước quyết định gần bằng 0. Biên bản từng lần nằm trong `docs/dien-tap/`, số đo gốc trong `data/azure/drills/`.

Bảng 3.3. Kết quả vòng chính thức trên Azure

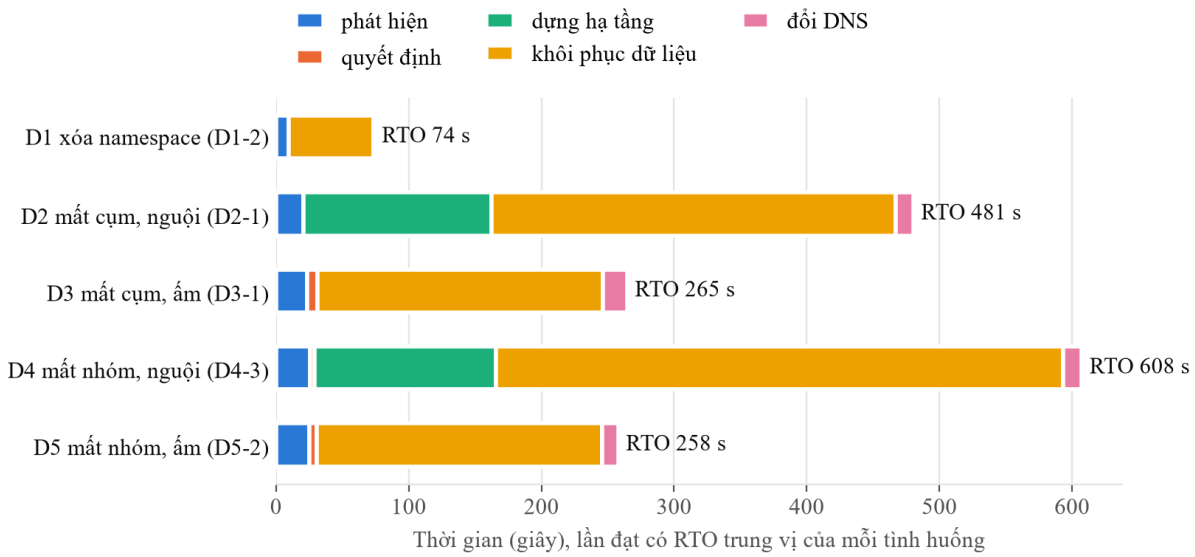
Lần	Tình huống	Kết quả	RTO (s)	RPO (s)	Ghi chú
D1-1	Xóa namespace	Đạt	73,8	3,0	
D1-2	Xóa namespace	Đạt	74,1	3,5	
D1-3	Xóa namespace	Đạt	74,6	2,6	
D2-1	Mất cụm chính, nguội	Đạt	481,1	4,8	
D2-2	Mất cụm chính, nguội	Đạt	470,9	46,3	
D2-3	Mất cụm chính, nguội	Đạt	579,3	49,1	Kiểm tra 146 s

Lần	Tình huống	Kết quả	RTO (s)	RPO (s)	Ghi chú
D3-1	Mất cụm chính, ấm	Đạt	265,2	8,4	
D3-2	Mất cụm chính, ấm	Đạt	265,7	18,5	
D3-3	Mất cụm chính, ấm	Đạt	256,1	5,8	
D4-1	Mất nhóm chính, nguội	Không đạt	-	-	Công cụ kiểm tra dừng vì một lần đọc ảnh bị treo 30 s
D4-2	Mất nhóm chính, nguội	Không đạt	-	-	Như D4-1
D4-3	Mất nhóm chính, nguội	Đạt	608,2	46,9	Từ lần này công cụ đọc lại ảnh khi lỗi mạng
D4-4	Mất nhóm chính, nguội	Đạt	645,5	47,3	
D4-5	Mất nhóm chính, nguội	Đạt	596,1	50,9	
D5-1	Mất nhóm chính, ấm	Đạt	265,7	19,8	
D5-2	Mất nhóm chính, ấm	Đạt	258,4	45,2	48 ảnh mất trong RPO ảnh
D5-3	Mất nhóm chính, ấm	Đạt	258,1	6,5	86 ảnh mất trong RPO ảnh

Ở 15 lần đạt, mã băm nội dung của bài viết khớp, không có ảnh hỏng, không có bài mất ngoài RPO, và đăng bài mới được. Hình 3.1 cho thấy RTO và RPO của từng lần; Hình 3.2 tách RTO của lần có RTO trung vị ở mỗi tình huống thành các bước.



Hình 3.1. RTO và RPO của các lần đạt ở vòng chính thức trên Azure. Mỗi chấm là một lần, vạch đen là trung vị, trong ngoặc là số lần đạt trên số lần chạy; tải 1 bài/s, bộ mẫu 1.026 bài. Đơn vị: giây



Hình 3.2. Thời gian từng bước của lần đạt có RTO trung vị ở mỗi tình huống, vòng chính thức trên Azure, một lần mỗi tình huống. Đơn vị: giây

Bảng 3.4. Thời gian từng bước của lần có RTO trung vị (s)

Lần	Phát hiện	Quyết định	Dựng hạ tầng	Khôi phục	Đổi DNS	Kiểm tra
D1-2	9,4	0,0	-	64,4	-	38,9
D2-1	20,7	0,1	141,8	304,7	12,9	40,2
D3-1	23,5	7,7	-	215,2	18,1	40,2
D4-3	25,7	3,6	136,6	427,6	13,7	38,1
D5-2	25,3	5,5	-	214,8	12,2	38,9

Hai lần D4-1 và D4-2 không có số đo vì công cụ kiểm tra dừng khi đọc thân phản hồi một ảnh bị treo trên đường truyền từ cụm về laptop. Nhật ký API cho thấy mọi yêu cầu ảnh đều được trả trong tối đa 217 ms, và đọc lại 3.312 lần không tái hiện được. Hai lần này vẫn được tính là không đạt. Ngoài ra, lỗi mạng tạm thời tới Azure làm hỏng ba lần dựng lại hạ tầng hoặc dựng cụm dự phòng trước bước gây sự cố; các lần này chưa bắt đầu diễn tập nên không tính, và lần dựng lại sau đều thành công.

3.4 Két chống xóa và khôi phục ngoài Azure

D6-1 (07/11/2026), chính sách WORM 7 ngày đã khóa cứng. Bốn cách xóa và ghi đè gói 410 MB đều bị từ chối: xóa và ghi đè bằng khóa tài khoản trả HTTP 409 `BlobImmutableDueToPolicy`, xóa và ghi đè bằng SAS của job két trả HTTP 403. Lệnh rút ngắn thời hạn WORM cũng bị từ chối. Gói giữ nguyên kích thước và thời điểm sửa.

D7-1 gồm hai phần:

- Từ két (07/11/2026): tải gói mới nhất về laptop, giải mã và khôi phục mất 1.201 s; số bài và mã băm nội dung khớp. Công cụ báo thiếu 86 ảnh; cả 86 ảnh đều của bài tạo trong 90 s trước sự cố của D5-3 và đã được tính là mất trong RPO ảnh ở lần đó. Chấm theo luật RPO ảnh: đạt.
- Từ máy chủ ngoài Azure (08/11/2026): lấy bản mới nhất từ restic mất 4 s, chép về laptop 221 s, khôi phục trên laptop không gọi tới Azure 59 s: 1.026 bài, mã băm nội dung khớp, không thiếu ảnh.

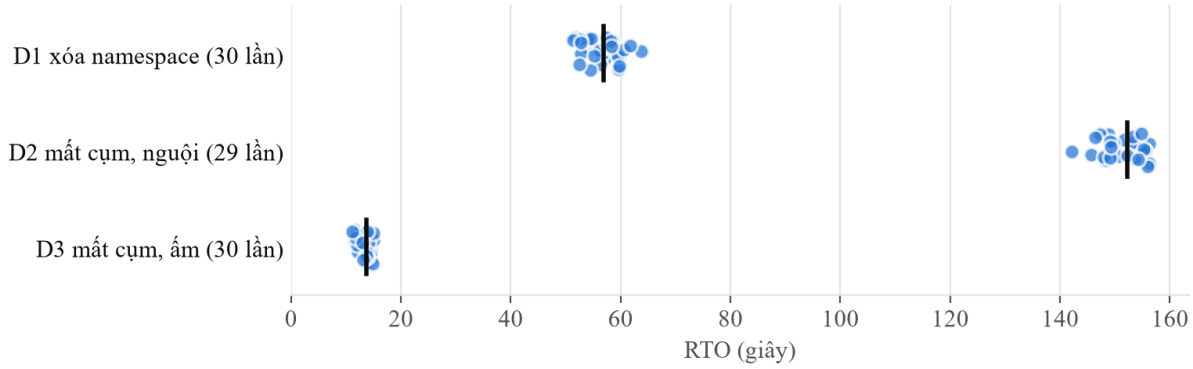
3.5 Bản thu nhỏ trên GitHub Actions

Lượt chạy ngày 08/11/2026 (workflow `kind-mini`, lượt 42434189591, commit `38bfddd`) lặp mỗi tình huống 30 lần, chia thành sáu job song song. Số liệu thô nằm trong `data/kind/2026-11-08-gh42434189591/`.

Bảng 3.5. Kết quả bản thu nhỏ trên GitHub Actions

Tình huống	Lần đạt	RTO trung vị (s)	RTO Q1 - Q3 (s)	RPO trung vị (s)	RTO nhỏ nhất - lớn nhất (s)
D1	30/30	56,9	54,1 - 58,4	0	51,4 - 63,8
D2	29/29	152,3	149,1 - 154,5	35,0	142,1 - 156,2
D3	30/30	13,7	12,1 - 14,1	35,0	11,1 - 15,0
D6	30/30	-	-	-	Xóa gói trong két luôn bị từ chối

Cả 119 lần chạy trọn vẹn đều đạt. Một lần D2 hỏng ở bước dựng cụm, trước bước gây sự cố, vì pod của Barman Cloud Plugin kẹt ở trạng thái `ContainerCreating` hơn 5 phút; nhật ký không lưu sự kiện Kubernetes nên chưa xác định được nguyên nhân.



Hình 3.3. RTO của bản thu nhỏ trên GitHub Actions, máy 4 nhân, 30 lần mỗi tình huống (D2: 29 lần). Mỗi chấm là một lần, vạch đen là trung vị. Không dựng máy ảo, không đổi DNS nên không so trực tiếp với RTO trên Azure. Đơn vị: giây

Ngày 10/11/2026, một thành viên khác chạy lại bản thu nhỏ trên laptop của mình theo hướng dẫn tái lập trong kho mã: D1 đạt, RTO 59,5 s, 1.096 trên 1.096 bài còn nguyên.

3.6 Chi phí thực tế

Theo Azure Cost Management (loại ActualCost, đọc ngày 08/11/2026), sáu nhóm tài nguyên của đồ án tốn 5,70 USD từ 25/09 đến hết 07/11/2026. Máy ảo B2als v2 chạy tổng 31,2 giờ, tốn 1,46 USD; truyền dữ liệu giữa hai vùng 1,08 USD; các thao tác đọc, ghi trên kho lưu trữ khoảng 2,14 USD; địa chỉ IP tĩnh 0,68 USD; ổ đĩa 0,34 USD. Riêng ngày chạy vòng chính thức (07/11) tốn 2,85 USD. Ngân sách Azure đặt 25 USD với cảnh báo email ở 50%, 80% và 100%; chi phí thật bằng khoảng 23% ngân sách. Số liệu thô nằm trong `data/azure/cost-2026-11-08.csv`.

CHƯƠNG 4: DIỄN GIẢI VÀ GIỚI HẠN

4.1 Kiểm chứng giả thuyết

- H1, dự phòng nguội dưới 60 phút: xác nhận. D2 mất 470,9 đến 579,3 s, D4 mất 596,1 đến 645,5 s, đều dưới 11 phút.
- H2, dự phòng ấm dưới 10 phút: xác nhận. D3 mất 256,1 đến 265,7 s, D5 mất 258,1 đến 265,7 s, đều dưới 4,5 phút.
- H3, dữ liệu mất không vượt RPO: xác nhận. RPO bài viết lớn nhất là 50,9 s (D4-5), dưới mục tiêu 300 s. Ảnh chỉ mất ở D5-2 và D5-3, đều là ảnh của bài đăng trong 300 s trước sự cố.
- H4, runbook chưa diễn tập sẽ lộ lỗi: xác nhận. Vòng thử có 7 lần không đạt trên 12 lần, và 10 thay đổi được ghi vào lịch sử runbook (phụ lục A). Không lỗi nào trong số đó nhìn thấy được khi chỉ đọc tài liệu: chúng chỉ lộ ra khi Azure trả về một ID dài hơn dự kiến, một máy ảo kẹt, hoặc một chính sách còn sót từ lần trước.
- H5, khôi phục từ két và bản ngoài Azure: xác nhận. Từ két mất 1.201 s, từ máy chủ ngoài Azure 59 s sau khi chép về; số bài và mã băm khớp.

H1 và H2 dùng mục tiêu của cả web lấy từ BIA, vốn rộng so với số đo, nên chỉ là phép thử yếu. Phép thử chặt hơn là RTO từng thành phần ở Bảng 2.1, xét ở mục 4.2.

Câu trả lời cho câu hỏi nghiên cứu: kế hoạch ban đầu không đạt mục tiêu. Ở lần chạy đầu tiên của mỗi tình huống, D1, D2 và D4 đều không đạt. Kế hoạch chỉ đạt mục tiêu sau khi đã được diễn tập và sửa. Lưu ý rằng kết quả vòng chính thức tốt hơn vòng thử không chỉ vì runbook được sửa, mà còn vì người chạy đã quen thao tác; hai yếu tố này không tách được với số lần chạy hiện có.

4.2 Sai lệch so với mục tiêu và điểm nghẽn

Mục tiêu của cả web đạt với biên rộng: RTO dự phòng nguội lớn nhất 645,5 s so với 3.600 s, RTO dự phòng ấm lớn nhất 265,7 s so với 600 s. Khoảng cách đó cho thấy mục tiêu 60 phút của dự phòng nguội được đặt rộng so với năng lực thực tế. Nếu vận hành thật, mục tiêu này có thể hạ xuống 15 phút: cả 6 lần đạt của D2 và D4 vẫn nằm trong mức đó.

Ở mức thành phần, dự phòng nguội đạt: bước khôi phục của lần có RTO trung vị mất 304,7 s ở D2 và 427,6 s ở D4 (Bảng 3.4), dưới mục tiêu 15 phút. Dự phòng ấm có một sai lệch. Bảng 2.1 đặt RTO 2 phút cho bài viết và ảnh khi chuyển sang dự phòng ấm, nhưng bước khôi phục của D3 và D5 kéo dài 213 đến 221 s, tức khoảng 3,6 phút. Vòng chính thức không đo riêng từng thành phần nên không khẳng định được thành phần nào vượt. Số đo khi chạy thử cho thấy riêng việc đổi vai Postgres chỉ mất 7 s (Bảng 3.1), nên phần lớn thời gian của bước nằm ở các việc còn lại: gỡ chính sách chép ảnh rồi chờ kho ảnh dự phòng nhận ghi (vòng thử ghi nhận Azure còn từ chối ghi thêm 1 đến 3 phút sau khi gỡ), khởi động lại API, và lấy một bản sao lưu đầy đủ trước khi đổi DNS. Đây là suy luận từ cấu trúc script, cần đo tách từng việc để xác nhận.

Điểm nghẽn chung của mọi tình huống là bước khôi phục dữ liệu: 87% RTO ở D1, 63% ở D2, 70% ở D4 và 81 đến 83% ở D3, D5 (Bảng 3.4). Riêng D4 có bước khôi phục dài hơn D2 khoảng 120 s vì phải chép khoảng 2.000 ảnh từ bản chép ở Nhật sang kho ảnh mới. Dựng hạ tầng ở dự phòng nguội tốn thêm 130 đến 159 s. Phát hiện sự cố mất 9 đến 31 s và đổi DNS mất khoảng 12 đến 25 s, đều nhỏ so với khôi phục.

Các cải tiến đề xuất, chưa được diễn tập lại:

- Lấy bản sao lưu đầy đủ sau khi đổi DNS thay vì trước, vì WAL vẫn được gửi liên tục trong lúc đó.
- Chép ảnh song song nhiều luồng khi mất kho ảnh chính.
- Tạo bản đầy đủ thường xuyên hơn để giảm lượng WAL phải phát lại khi chuyển sang dự phòng nguội.

Bước kiểm tra (37 đến 146 s) không nằm trong RTO vì nó chạy sau khi dịch vụ đã phục vụ lại; lần D2-3 có bước kiểm tra 146 s và chưa rõ nguyên nhân.

4.3 Runbook sau diễn tập

Mọi lỗi ở vòng thử đều dẫn tới một thay đổi trong runbook hoặc script, ghi vào lịch sử thay đổi ở cuối runbook (phụ lục A). Có bốn nhóm lỗi:

- Lỗi giả định về trạng thái: cụm mới chưa có bản đầy đủ (D1-1), bước chép ảnh chờ cả những ảnh không bao giờ tới (D4-1), chính sách chép ảnh còn sót từ lần trước (D2-3, D4-2). Sửa bằng cách cho script tự tạo trạng thái cần thiết thay vì giả định nó đã có.

- Lỗi về cách Azure trả kết quả: ID tài nguyên đầy đủ thay vì tên, ký tự xuống dòng thừa từ `az` trên Windows, kho lưu trữ còn từ chối ghi vài phút sau khi gỡ chính sách (D2-2). Sửa bằng cách so đúng phần cuối của ID và chờ tới khi ghi thử thành công.
- Lỗi viết script: câu lệnh jsonpath chọn bản Velero sai cú pháp (D1-2). Sửa cú pháp; lần D1-3 sau đó đạt.
- Lỗi của hạ tầng: máy ảo kẹt ở `starting` (D2-1). Không sửa được ở phía nhóm; runbook thêm bước xóa cưỡng bức và dựng lại.

Ngoài runbook, luật chấm của công cụ kiểm tra được sửa một lần (D4-3) để phân biệt ảnh mất trong RPO với ảnh hỏng. Thay đổi này được ghi rõ và áp dụng cho mọi lần sau, không áp ngược để biến lần không đạt thành đạt, trừ D4-3 được chấm lại trên đúng dữ liệu đã khôi phục.

4.4 Kế hoạch kiểm thử lại

- Sau vòng chính thức: tình huống nào trượt mục tiêu thì sửa điểm nghẽn rồi chạy lại đúng tình huống đó. Ở vòng này không tình huống nào trượt mục tiêu; D4 có hai lần không đạt do công cụ kiểm tra và đã được chạy thêm cho đủ ba lần đạt.
- Trước khi gắn thẻ phiên bản nghiệm thu: chạy lại các tình huống có script bị sửa. Script khôi phục và chuyển cụm (restore-namespace.sh, failover-cold.sh, failover-warm.sh) không đổi từ commit 850ac35 của vòng chính thức tới thẻ v1.1; phần đổi sau đó là script két, bản sao ngoài Azure, bản thu nhỏ và tài liệu. Vì vậy chỉ chạy lại D1 trên bản thu nhỏ (mục 3.5); D3, D6 và D7 chưa chạy lại trên đúng thẻ này vì cụm Azure đã tắt từ 07/11/2026.
- Khi đổi phiên bản công cụ (k3s, CNPG, Velero), hạ tầng hoặc runbook: chạy lại các tình huống bị ảnh hưởng.
- Tự động: diễn tập dự phòng nguội mỗi ngày trên GitHub Actions (mục 2.7) là kiểm thử định kỳ, kèm kiểm tra sao lưu mỗi giờ.
- Nếu vận hành thật: diễn tập toàn bộ mỗi quý, có ít nhất một lần không báo trước và một lần do người không viết runbook chạy, vì hai yếu tố này chưa được đo trong đồ án.

4.5 Chi phí sẵn sàng và lựa chọn chiến lược

Bảng 4.1 ước tính chi phí để giữ mỗi chiến lược sẵn sàng trong một tháng (730 giờ), theo bảng giá bán lẻ Azure lấy qua Azure Retail Prices API [33] ngày 07/10/2026.

Bảng 4.1. Chi phí sẵn sàng mỗi tháng và RTO đo được

Chiến lược	Phải trả thêm để sẵn sàng	USD/tháng	RTO trung vị đo được
Dự phòng nguội	Chỉ lưu trữ bản sao lưu	Khoảng 1	481 s (D2), 608 s (D4)
Dự phòng ấm	Một cụm dự phòng chạy suốt: máy, ổ đĩa và IP khoảng 0,059 USD/giờ, cộng lưu trữ	Khoảng 44	265 s (D3), 258 s (D5)

Dự phòng ấm rút RTO đi khoảng 216 s khi mất cụm và khoảng 350 s khi mất cả nhóm tài nguyên, với giá khoảng 43 USD mỗi tháng. Dự phòng nguội đã đạt mục tiêu 60 phút với biên rộng, nên với web quy mô nhỏ như đồ án, dự phòng nguội là lựa chọn hợp lý. Dự phòng ấm chỉ đáng chọn khi thiệt hại do ngừng thêm khoảng 4 đến 6 phút mỗi lần sự cố, nhân với số lần sự cố mỗi tháng, lớn hơn khoảng 43 USD.

Cần lưu ý rằng cụm dự phòng ấm trong đồ án cũng ở `koreacentral`. Nếu mất cả vùng thật, cụm này mất theo. Một phương án dự phòng ấm thật cho mất vùng phải đặt cụm dự phòng ở vùng thứ hai, chi phí tương đương nhưng thêm phí truyền dữ liệu giữa hai vùng.

4.6 Rủi ro bảo mật còn lại

- Khóa age dành cho diễn tập tự động và các mã chỉ đọc nằm trong mục Secrets của kho GitHub. Ai có quyền quản trị kho GitHub thì đọc được bản trong két. Dữ liệu là dữ liệu tổng hợp nên chấp nhận được; với dữ liệu thật, khóa này phải tách sang một hệ thống riêng.
- Khóa bí mật khôi phục và mật khẩu restic nằm trong trình quản lý mật khẩu của một người. Mất khóa này thì không giải mã được két và bản ngoài Azure.
- Máy chủ ngoài Azure do một cá nhân ngoài nhóm cho mượn. Gói lưu ở đó đã mã hóa và máy không giữ khóa giải mã, nhưng nhóm không kiểm soát việc máy còn chạy hay không.
- Cổng quản trị (SSH 22, Kubernetes API 6443) mở cho địa chỉ IP của nhóm. Địa chỉ IP nhà mạng thay đổi thì phải cập nhật Terraform.

- Kho mã là kho công khai. Khóa và mã bí mật nằm ngoài kho (thư mục riêng trên máy người trực, mục Secrets của GitHub), nhưng mỗi commit vẫn phải được rà trước khi đẩy lên để không lọt khóa.

4.7 Giới hạn của đồ án

- Mất vùng chỉ là giả lập: tài khoản sinh viên chỉ cho tạo máy ảo ở `koreacentral`, nên D4 và D5 xóa cả nhóm tài nguyên chính chứ không mất vùng thật.
- Mọi lần ở vòng chính thức đều được báo trước và do một người chạy bằng công cụ tự động, với bước quyết định gần bằng 0. Lần không báo trước và lần do thành viên khác chạy theo runbook chưa thực hiện, nên thời gian phát hiện và quyết định của người trực thật chưa được đo.
- RTO từng thành phần chưa được đo riêng; sai lệch của dự phòng ấm ở mục 4.2 mới chỉ xác định ở mức cả bước khôi phục.
- Mỗi tình huống trên Azure chỉ có ba lần đạt, đủ để so với mục tiêu nhưng không đủ để suy luận thống kê.
- Máy dòng B dùng cơ chế tín dụng CPU: hết tín dụng thì máy chỉ chạy ở mức hiệu năng nền [23]; máy mới dựng và máy chạy lâu có thể nhanh chậm khác nhau, làm thời gian dựng hạ tầng dao động.
- Bản thu nhỏ chạy trên một loại máy GitHub duy nhất, dùng SeaweedFS và không đổi DNS. Số đo của nó cho thấy quy trình ổn định khi lặp lại, không dùng để ước lượng RTO trên Azure.
- Tầng thứ ba chỉ có một máy chủ, không có bản dự phòng cho chính nó; máy chủ từng khởi động lại ngoài kế hoạch ngày 08/11/2026.
- Khôi phục từ két mất khoảng 20 phút, phần lớn là tải gói khoảng 410 MB từ Nhật về laptop.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án dựng một web đăng bài kèm ảnh trên Azure, lập kế hoạch phục hồi sau thảm họa ba tầng (kho sao lưu ở Nhật, két chống xóa có mã hóa, máy chủ ngoài Azure) và kiểm chứng kế hoạch bằng diễn tập có đo đạc thay vì suy đoán.

Câu trả lời cho câu hỏi nghiên cứu là: kế hoạch chưa diễn tập không đạt mục tiêu. Vòng thử có 7 trên 12 lần không đạt, mỗi lần lộ một lỗi mà việc đọc tài liệu không phát hiện được. Sau khi sửa, vòng chính thức có 15 lần đạt trên 17 lần chạy D1 đến D5, và mọi lần đạt đều trong mục tiêu: RTO dự phòng nguội lớn nhất 645,5 s (mục tiêu 60 phút), RTO dự phòng ấm lớn nhất 265,7 s (mục tiêu 10 phút), RPO bài viết lớn nhất 50,9 s (mục tiêu 5 phút). Hai lần không đạt là do công cụ kiểm tra.

Két chống xóa đã khóa cứng từ chối mọi cách xóa, ghi đè và rút ngắn thời hạn, kể cả với khóa tài khoản. Dữ liệu khôi phục được hoàn toàn từ két và từ máy chủ ngoài Azure mà không cần kho sao lưu chính. Bản thu nhỏ chạy 119 lần trọn vẹn đều đạt với khoảng tứ phân vị của RTO dưới 6 s, cho thấy quy trình lặp lại ổn định.

Toàn bộ quá trình thử nghiệm và diễn tập trên Azure tốn 5,70 USD. Dự phòng ấm rút RTO từ khoảng 8 đến 10 phút xuống khoảng 4,4 phút nhưng tốn khoảng 44 USD mỗi tháng so với khoảng 1 USD của dự phòng nguội; với web quy mô nhỏ như đồ án, dự phòng nguội là đủ.

5.2 Hướng phát triển

- Chạy lần gây sự cố không báo trước và lần do thành viên khác tự làm theo runbook, để đo khả năng của người trực chứ không chỉ của script.
- Rút ngắn bước khôi phục: lấy bản đầy đủ sau khi đổi DNS, chép ảnh song song, tạo bản đầy đủ thường xuyên hơn.
- Thử lại có giới hạn khi tải gói từ két và chạy Terraform, hai chỗ hay lỗi mạng.
- Làm quy trình quay về cụm chính (failback); đặt cụm dự phòng ở vùng thứ hai và thêm một nơi lưu thứ hai ngoài Azure để tầng thứ ba không phụ thuộc một máy chủ.

TÀI LIỆU THAM KHẢO

[1] Cloud Native Computing Foundation, “CNCF cloud native definition v1.1,” CNCF TOC repository. Accessed: Nov. 11, 2026. [Online]. Available: https://github.com/cncf/toc/blob/main/DEFINITION.md

[2] HashiCorp, “Terraform documentation,” HashiCorp Developer. Accessed: Nov. 11, 2026. [Online]. Available: https://developer.hashicorp.com/terraform/docs

[3] R. Blum and R. Singh, “Data integrity: what you read is what you wrote,” in *Site Reliability Engineering: How Google Runs Production Systems*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Sebastopol, CA: O’Reilly Media, 2016. Accessed: Nov. 11, 2026. [Online]. Available: https://sre.google/sre-book/data-integrity/

[4] M. Swanson, P. Bowen, A. W. Phillips, D. Gallup, and D. Lynes, “Contingency planning guide for federal information systems,” National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-34 Rev. 1, May 2010. doi: 10.6028/NIST.SP.800-34r1.

[5] Amazon Web Services, “Disaster recovery of workloads on AWS: recovery in the cloud,” Amazon Web Services, AWS Whitepaper, 2021. Accessed: Nov. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-workloads-on-aws.html

[6] The PostgreSQL Global Development Group, “Continuous archiving and point-in-time recovery (PITR),” PostgreSQL documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/current/continuous-archiving.html

[7] P. Krogh, *The DAM Book: Digital Asset Management for Photographers*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2009.

[8] P. Ruggiero and M. A. Heckathorn, “Data backup options,” United States Computer Emergency Readiness Team, 2012. Accessed: Nov. 11, 2026. [Online]. Available: https://www.cisa.gov/sites/default/files/publications/data_backup_options.pdf

[9] Cybersecurity and Infrastructure Security Agency, “#StopRansomware guide,” Cybersecurity and Infrastructure Security Agency, Joint Ransomware Task Force guide, Oct. 2023. Accessed: Nov. 11, 2026. [Online]. Available: https://www.cisa.gov/resources-tools/resources/stopransomware-guide

[10] Microsoft, “About Azure VM backup,” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/backup/backup-azure-vms-introduction

[11] Velero Authors, “Backup hooks,” Velero documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://velero.io/docs/v1.17/backup-hooks/

[12] The PostgreSQL Global Development Group, “pg_dump,” PostgreSQL documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/current/app-pgdump.html

[13] CloudNativePG Contributors, “Backup,” CloudNativePG documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://cloudnative-pg.io/docs/current/backup/

[14] Microsoft, “Immutable storage for blob data overview,” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/storage/blobs/immutable-storage-overview

[15] Microsoft, “Azure Storage encryption for data at rest,” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/storage/common/storage-service-encryption

[16] F. Valsorda, *age: a simple, modern and secure file encryption tool*. GitHub. Accessed: Nov. 11, 2026. [Online]. Available: https://github.com/FiloSottile/age

[17] T. Grance, T. Nolan, K. Burke, R. Dudley, G. White, and T. Good, “Guide to test, training, and exercise programs for IT plans and capabilities,” National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-84, Sep. 2006. doi: 10.6028/NIST.SP.800-84.

[18] Lê Quốc Tuấn, “Bài giảng Điện toán đám mây, Chương 4: An toàn, bảo mật và quản trị điện toán đám mây.” Trường Đại học Giao thông vận tải Thành phố Hồ Chí Minh, Thành phố Hồ Chí Minh, 2026.

[19] CloudNativePG Contributors, “Replica clusters,” CloudNativePG documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://cloudnative-pg.io/docs/current/replica_cluster/

[20] Microsoft, “Object replication overview,” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/storage/blobs/object-replication-overview

[21] K3s Project Authors, “K3s: lightweight Kubernetes,” K3s documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://docs.k3s.io/

[22] Microsoft, “Use system node pools in Azure Kubernetes Service (AKS),” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/aks/use-system-pools

[23] Microsoft, “Basv2 size series,” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/virtual-machines/sizes/general-purpose/basv2-series

[24] MinIO, “minio/minio: MinIO object store repository (archived),” GitHub. Accessed: Nov. 11, 2026. [Online]. Available: https://github.com/minio/minio

[25] Helm Authors, “Helm documentation,” Helm. Accessed: Nov. 11, 2026. [Online]. Available: https://helm.sh/docs/

[26] Let’s Encrypt, “Rate limits,” Let’s Encrypt. Accessed: Nov. 11, 2026. [Online]. Available: https://letsencrypt.org/docs/rate-limits/

[27] CloudNativePG Contributors, “Barman Cloud CNPG-I plugin,” CloudNativePG. Accessed: Nov. 11, 2026. [Online]. Available: https://cloudnative-pg.io/plugin-barman-cloud/

[28] A. Pennarun, “How Tailscale works,” Tailscale blog. Accessed: Nov. 11, 2026. [Online]. Available: https://tailscale.com/blog/how-tailscale-works

[29] restic Authors, “restic documentation,” restic. Accessed: Nov. 11, 2026. [Online]. Available: https://restic.readthedocs.io/

[30] Microsoft, “Grant limited access to data with shared access signatures (SAS),” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/azure/storage/common/storage-sas-overview

[31] Kubernetes SIGs, “kind,” kind documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://kind.sigs.k8s.io/

[32] The Kubernetes Authors, “Namespaces,” Kubernetes documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/

[33] Microsoft, “Azure retail prices REST API overview,” Microsoft Learn. Accessed: Nov. 11, 2026. [Online]. Available: https://learn.microsoft.com/rest/api/cost-management/retail-prices/azure-retail-prices

PHẦN BỔ SUNG SẢN PHẨM

1. Kho mã nguồn: https://github.com/qivarnq3g/cloud-native-disaster-recovery (thẻ phiên bản nghiệm thu `v1.1`)

2. Video demo: https://youtu.be/iPRazYU_bQM

3. Slide: https://www.canva.com/design/DAHalYk4XpM/-IFevy7-a-TovcQ0UpPvKQ/view

4. Web: https://dr.qivarn.id.vn (chỉ chạy trong các buổi diễn tập; cụm Azure đã tắt từ 07/11/2026 nên trang hiện báo máy chủ đang được khôi phục)

5. Phân công (như trong README của kho mã):

Hoàng Việt Quang: web, hai cụm Kubernetes, sao lưu, script đo và kiểm tra, diễn tập, các chương kỹ thuật.

Lê Thị Lan Hương: lý thuyết, đánh giá ảnh hưởng, nháp runbook, tài liệu tham khảo.

Nguyễn Minh Thư: danh sách kiểm tra sau khôi phục, bấm giờ và biên bản diễn tập, ghép báo cáo, slide, video.

PHỤ LỤC A: RUNBOOK SAU DIỄN TẬP

Bản đầy đủ là `docs/runbook.md` trong kho mã. Mọi lệnh chạy trong Git Bash ở thư mục gốc của kho; `<run_id>` là mã lần diễn tập, hoặc `incident-<ngày giờ>` khi có sự cố thật.

A.1 Khi nào kích hoạt

- `https://api.dr.qivarn.id.vn/healthz` lỗi hoặc không trả lời 5 lần liên tiếp, mỗi lần cách 1 s, trong khi máy người trực vẫn vào được `https://www.gstatic.com/generate_204`.
- Hoặc người trực thấy namespace `dr-api`, máy `vm-dr-primary` hay nhóm `rg-dr-primary` đã bị xóa.

A.2 Chọn cách khôi phục

Bảng A.1. Chọn cách khôi phục theo tình huống

Tình huống	Cách	Lệnh
Mất namespace `dr-api`, máy chính còn	Tại chỗ	`restore-namespace.sh <run_id> <giờ sự cố trừ 1 s>`
Mất máy chính, chưa có cụm dự phòng	Nguội	`failover-cold.sh <run_id> standby`
Mất máy chính, cụm dự phòng đang chạy	Ấm	`failover-warm.sh <run_id>`
Mất cả nhóm `rg-dr-primary`	Dự phòng nguội có chép ảnh, hoặc dự phòng ấm	`DR_IMAGES_FROM_COPY=true failover-cold.sh <run_id> standby`, hoặc `failover-warm.sh <run_id>`

A.3 Các bước

- Phát hiện: ghi giờ sự cố theo UTC.
- Quyết định: người có quyền kích hoạt gửi tin "bắt đầu sự cố" vào nhóm Zalo.
- Khôi phục bằng lệnh ở Bảng A.1. Script ghi thời gian từng bước vào `data/azure/drills/<run_id>/steps.csv`.
- Nếu máy ảo mới kẹt ở `starting` quá 10 phút: xóa cưỡng bức máy và ổ đĩa (`az vm delete ... --force-deletion true`, `az disk delete ...`) rồi chạy lại lệnh chuyển sang dự phòng nguội.

- Kiểm tra theo Bảng A.2, rồi gửi tin "đã chuyển sang dự phòng" khi DNS đã đổi và "đã khôi phục xong" khi kiểm tra đạt.
- Dựng lại trạng thái ban đầu sau diễn tập theo thứ tự trong `docs/runbook.md`: dọn ổ đĩa mồ côi, tắt phiên ở cụm dự phòng, dựng lại kho ảnh chính nếu cả nhóm đã mất, bật phiên ở cụm chính, dựng lại chính sách chép ảnh, đưa dữ liệu về bộ mẫu.

A.4 Danh sách kiểm tra sau khôi phục

Bảng A.2. Danh sách kiểm tra sau khôi phục

Mục	Cách kiểm tra	Đạt khi
API phục vụ qua tên miền	Gọi `/healthz` qua `api.dr.qivarn.id.vn`	Trả 200 từ cụm mới
Số bài	So danh sách bài trong sổ của script đo với cơ sở dữ liệu	Không thiếu bài nào ngoài RPO 5 phút
Nội dung bài	So mã băm nội dung của toàn bộ bài	Khớp
Ảnh	Tải lại từng ảnh, so mã băm SHA-256 với mã lưu trong bài	Không ảnh hỏng; ảnh thiếu chỉ thuộc bài đăng trong 300 s trước sự cố
Chức năng	Đăng một bài mới kèm ảnh	Đăng được, bài hiện trong danh sách
Dữ liệu ngoài RPO	Liệt kê riêng bài và ảnh mất	Ghi vào biên bản

A.5 Lịch sử thay đổi

Bảng A.3. Lịch sử thay đổi runbook sau diễn tập

Ngày	Sau lần	Thay đổi
31/10/2026	trial-D1-1	Lấy bản đầy đủ ngay khi có server Postgres mới; bật phiên lấy thêm bản Velero; khôi phục namespace chỉ từ bản Velero của phiên đang chạy
31/10/2026	trial-D1-2	Sửa câu jsonpath chọn bản Velero
31/10/2026	trial-D2-1	Thêm bước xử lý máy ảo kẹt ở `starting` và ổ đĩa còn sót
31/10/2026	trial-D2-2	Gỡ chính sách chép ảnh theo ID đầy đủ, chờ kho ảnh dự phòng nhận ghi
31/10/2026	trial-D2-3	Thêm `restore-replication.sh` vào bước dựng lại
31/10/2026	trial-D3-1	Chuyển sang dự phòng ấm tạo lại lịch sao lưu đầy đủ và lịch Velero
04/11/2026	trial-D4-1	Chép ảnh từ bản chép chỉ chờ các ảnh đang chép dở
04/11/2026	trial-D4-2	Chuyển sang dự phòng nguội gỡ chính sách chép ảnh của kho dự phòng trước khi chép ảnh

Ngày	Sau lần	Thay đổi
04/11/2026	trial-D4-3	Ảnh thiếu của bài đăng trong 5 phút trước sự cố tính là mất trong RPO; thêm `rebuild-primary-storage.sh` khi mất cả nhóm chính
07/11/2026	official-D4-1	Sau `rebuild-primary-storage.sh` bỏ qua `restore-replication.sh` khi dựng lại

PHỤ LỤC B: BIÊN BẢN VÒNG THỬ

Biên bản đầy đủ của từng lần nằm trong `docs/dien-tap/trial-*.md`, số đo gốc trong `data/azure/drills/trial-*/`. RTO tính từ lúc gây sự cố tới lúc API phục vụ lại qua tên miền.

Bảng B.1. Kết quả vòng thử trên Azure

Lần	Tình huống	Kết quả	RTO (s)	RPO (s)	Ghi chú
D1-1	Xóa namespace	Không đạt	-	-	Khôi phục hỏng sau 910 s: thiếu bản đầy đủ, Velero lấy nhầm bản cũ
D1-2	Xóa namespace	Không đạt	-	-	Khôi phục hỏng sau 8 s: lỗi cú pháp jsonpath
D1-3	Xóa namespace	Đạt	74,0	4,2	Khôi phục 64,1 s
D2-1	Mất cụm chính, nguội	Không đạt	-	-	Dựng hạ tầng hỏng sau 2.490 s: `VMStartTimedOut`
D2-2	Mất cụm chính, nguội	Không đạt	416,5	6,4	Dữ liệu đủ nhưng đăng bài trả 503: `OrsPolicyConflict`
D2-3	Mất cụm chính, nguội	Không đạt	433,6	42,8	50 ảnh hỏng: kho dự phòng không nhận ảnh suốt phiên
D2-4	Mất cụm chính, nguội	Đạt	451,5	41,6	Dựng 137,7 s, khôi phục 275,8 s, DNS 16,5 s
D3-1	Mất cụm chính, ấm	Đạt	247,2	19,5	Khôi phục 214,6 s, DNS 11,2 s
D4-1	Mất nhóm chính, nguội	Không đạt	-	-	Bước chép ảnh hỏng sau 1.616 s
D4-2	Mất nhóm chính, nguội	Không đạt	-	-	Bước khôi phục hỏng sau 301 s: chính sách chép ảnh cũ chặn ghi
D4-3	Mất nhóm chính, nguội	Đạt (chấm lại)	588,4	18,1	Chép 2.022 ảnh trong 294 s; 16 ảnh mất trong RPO ảnh
D5-1	Mất nhóm chính, ấm	Đạt	262,3	47,0	Khôi phục 216,3 s, DNS 13,2 s

Trước khi khóa cứng chính sách, D6 và D7 được thử lần đầu ngày 04/11/2026. D6: bốn cách xóa và ghi đè gói 410 MB đều bị từ chối, nhưng khi chính sách chưa khóa cứng, người có khóa tài khoản vẫn rút ngắn được thời hạn; điểm yếu này được đóng bằng cách khóa cứng trước vòng chính thức (mục 3.4). D7: gói tải từ két bằng SAS chỉ đọc, giải mã bằng khóa `age` và khôi phục vào Postgres 18.6 chạy bằng Docker trên laptop trong 298 s, đủ 1.073 bài và 2.084 ảnh, mã băm khớp.